A PROJECT REPORT
ON
AI–BASED CROP RECOMMENDATION FOR FARMERS

Submitted by
Nandhitha PT
Register Number: 922524148069
Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)
V.S.B. Engineering College
Karur

TABLE OF CONTENTS

ABSTRACT

Agriculture plays a vital role in India's economy, supporting the livelihood of a significant portion of the population. However, selecting the most suitable crop remains a major challenge due to variations in soil properties, climatic conditions, and limited access to scientific decision-making tools. Traditional crop selection methods often rely on experience and regional practices, which may not always produce optimal results under changing environmental conditions. To address this issue, the **Smart Farmer: AI-Based Smart Crop Recommendation System** has been developed to provide accurate, data-driven crop recommendations using machine learning.

The proposed system is a full-stack web application that integrates modern web technologies with artificial intelligence to assist farmers in making informed agricultural decisions. It analyzes essential soil and environmental parameters, including **Nitrogen (N), Phosphorus (P), Potassium (K), soil pH, temperature, humidity, and rainfall**, to predict the most suitable crop for cultivation. The prediction is performed using a trained **Extra Trees Classifier**, selected for its high prediction accuracy, robustness, and ability to model complex relationships among agricultural features.

The application is built using **React.js** with **Tailwind CSS** for a responsive and user-friendly interface, while **Node.js** and **Express.js** provide secure RESTful APIs for application logic and user authentication. A **Flask** service hosts the trained machine learning model and processes prediction requests, whereas **PostgreSQL** is used to store user information and recommendation history. In addition to recommending the best crop, the system presents the **top five suitable crops**, along with confidence scores, crop category, water requirements, growth duration, and market demand to support better decision-making.

The project follows a structured software development methodology involving data preprocessing, model training, backend integration, database management, and frontend implementation. The integrated system was tested across all functional modules, including user registration, login, crop prediction, dashboard, and recommendation history, ensuring reliable performance and a seamless user experience.

By combining machine learning with modern web technologies, the Smart Farmer system promotes precision agriculture through scientifically based crop recommendations. The proposed solution aims to improve crop productivity, optimize resource utilization, and reduce uncertainties in crop selection. Furthermore, the application provides a scalable foundation for future enhancements such as real-time weather integration, fertilizer recommendations, IoT-enabled soil monitoring, satellite-based analysis, and mobile application support, contributing toward sustainable and technology-driven agriculture.

OBJECTIVES

The **Smart Farmer: AI-Based Smart Crop Recommendation System** is developed to provide an intelligent and reliable solution for crop selection by combining machine learning with modern web technologies. The project aims to assist farmers in making informed agricultural decisions based on scientific analysis of soil and climatic conditions. The objectives are classified into **primary** and **secondary** objectives.

Primary Objectives

- To develop an AI-based crop recommendation system that predicts the most suitable crop using soil nutrients (Nitrogen, Phosphorus, Potassium) and environmental parameters such as temperature, humidity, rainfall, and soil pH.

- To build and integrate an **Extra Trees Classifier** model capable of providing accurate and reliable crop predictions.

- To design and implement a full-stack web application using **React.js**, **Tailwind CSS**, **Node.js**, **Express.js**, **Flask**, and **PostgreSQL** for seamless interaction between users and the machine learning model.

- To provide users with the best crop recommendation along with the **top five suitable crops**, confidence scores, crop category, water requirements, growth duration, and market demand.

- To implement secure user authentication and maintain prediction history for personalized access and future reference.

Secondary Objectives

- To develop a responsive, user-friendly, and accessible interface that functions efficiently across multiple devices.

- To establish a scalable and modular system architecture that supports future enhancements and technology integration.

- To enable secure communication between the frontend, backend, database, and machine learning service through RESTful APIs.

- To ensure reliable data management, efficient system performance, and ease of maintenance.

- To provide a foundation for future features such as real-time weather integration, fertilizer recommendation, crop disease detection, IoT-based soil monitoring, and market price prediction.

- To promote precision agriculture by encouraging data-driven decision-making, improving crop productivity, optimizing resource utilization, and supporting sustainable farming practices.

SCOPE OF THE PROJECT

Present Scope

The **Smart Farmer: AI-Based Smart Crop Recommendation System** is designed as a web-based application that assists farmers in selecting the most suitable crop based on soil and climatic conditions. The current system accepts seven key agricultural parameters—**Nitrogen (N), Phosphorus (P), Potassium (K), soil pH, temperature, humidity, and rainfall**—and predicts the most appropriate crop using a trained **Extra Trees Classifier** model.

The application provides essential functionalities, including **user registration and secure login**, an interactive dashboard, a crop recommendation form, real-time machine learning prediction through a **Flask API**, and a detailed recommendation page displaying the predicted crop, confidence score, water requirement, growth duration, market demand, and the top five recommended crops. Additionally, the system stores user information and prediction history in a **PostgreSQL** database, allowing users to review their previous recommendations. The application is developed using **React.js**, **Tailwind CSS**, **Node.js**, and **Express.js**, ensuring a responsive user interface and efficient communication between the frontend, backend, database, and machine learning model. The present implementation is intended to demonstrate the practical integration of machine learning into a full-stack web application for precision agriculture.

Future Applicability in Agriculture

The Smart Farmer system has been developed with a scalable and modular architecture, enabling future enhancements to support advanced precision farming applications. The platform can be extended by integrating **real-time weather forecasting APIs** to provide more accurate crop recommendations based on current climatic conditions. Future versions may also incorporate **IoT-based soil sensors** for automatic collection of soil nutrient and moisture data, reducing the need for manual input.

Additional enhancements include **fertilizer recommendation**, **crop disease detection using computer vision**, **satellite imagery and remote sensing analysis**, and **market price prediction** to assist farmers in maximizing profitability. The system can also be expanded with **multilingual support**, **voice-based assistance**, and **mobile application development** to improve accessibility for farmers across different regions. Furthermore, integration with government agricultural schemes and subsidy information can provide farmers with valuable financial and policy-related support. These future improvements have the potential to transform the Smart Farmer application into a comprehensive, intelligent, and scalable digital agriculture platform that promotes sustainable farming and data-driven decision-making.

LITERATURE SURVEY

The integration of **Artificial Intelligence (AI)** and **Machine Learning (ML)** in agriculture has significantly improved decision-making processes by providing intelligent solutions for crop selection, yield prediction, disease detection, and resource management. Crop recommendation systems have become an important research area as they help farmers select the most suitable crop based on soil properties and environmental conditions, thereby improving agricultural productivity and sustainability.

Several researchers have developed crop recommendation systems using machine learning algorithms such as **Decision Tree, Naïve Bayes, Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (KNN),** and **Extra Trees Classifier**. These systems primarily utilize agricultural parameters, including soil nutrients, temperature, humidity, rainfall, and pH, to predict suitable crops. While many of these approaches have demonstrated satisfactory prediction accuracy, most of them are limited to experimental models or research prototypes and lack user-friendly web-based implementations for practical use by farmers.

Artificial Intelligence has also been widely applied in modern agriculture beyond crop recommendation. Recent studies have focused on **crop yield prediction**, **soil quality assessment**, **crop disease detection using image processing**, **weather forecasting**, **precision irrigation**, and **market price analysis**. These applications enable farmers to optimize agricultural practices, reduce production costs, and increase crop yield through data-driven decision-making. The rapid advancement of cloud computing, Internet of Things (IoT), and machine learning has further accelerated the development of intelligent farming solutions.

Among various machine learning techniques, **ensemble learning algorithms** have consistently produced superior results for agricultural prediction problems. Models such as **Random Forest** and **Extra Trees Classifier** offer higher prediction accuracy, improved generalization, and better resistance to overfitting compared to individual classifiers. The **Extra Trees Classifier** is particularly effective because it introduces additional randomness during tree construction, allowing it to capture complex relationships within agricultural datasets while maintaining computational efficiency. Based on comparative evaluation, the Extra Trees Classifier was selected for the proposed system due to its high accuracy and reliable performance in crop prediction.

The proposed **Smart Farmer: AI-Based Smart Crop Recommendation System** extends existing research by integrating the trained machine learning model into a complete full-stack web application. The system combines **React.js** with **Tailwind CSS** for the frontend, **Node.js** and **Express.js** for backend services, **Flask** for machine learning model deployment, and **PostgreSQL** for data management. Unlike many existing systems that provide only a single prediction, the proposed application presents the **top five suitable crops**, confidence scores, water requirements, growth duration, market demand, and prediction history, thereby offering a more comprehensive decision-support system for farmers. This integration of modern web technologies with machine learning makes the Smart Farmer application a practical, scalable, and user-friendly solution for precision agriculture.

EXISTING SYSTEM

Traditionally, crop selection has been based on farmers' experience, regional agricultural practices, and seasonal observations rather than scientific analysis. Farmers often depend on inherited knowledge, local expertise, and recommendations from agricultural extension officers to determine which crop should be cultivated. Although these conventional methods have been practiced successfully for many years, they do not adequately consider the complex relationship between soil nutrients, climatic conditions, and crop suitability. As a result, farmers may select crops that are not optimal for their land, leading to lower productivity and inefficient utilization of available resources.

Most existing crop recommendation approaches are either manual or limited to basic advisory systems. While some agricultural organizations provide recommendations based on soil testing, these services are not always easily accessible, personalized, or capable of delivering real-time recommendations. In many rural areas, farmers continue to rely on traditional decision-making practices without the support of intelligent technologies, making crop selection vulnerable to changing environmental conditions and market uncertainties.

Limitations of the Existing System

- Crop selection is primarily based on traditional knowledge and personal experience rather than scientific data analysis.
- Soil nutrient information is not effectively utilized to provide personalized crop recommendations.
- Existing advisory systems often lack real-time, intelligent decision-support capabilities.
- Most traditional methods do not consider multiple environmental factors such as temperature, humidity, rainfall, and soil pH simultaneously.
- Farmers receive limited information regarding alternative crop options, confidence levels, water requirements, and market demand.
- Manual record-keeping of previous crop recommendations and farming decisions is inefficient and difficult to maintain.
- Existing systems have limited accessibility in rural areas and often require expert consultation, making them less convenient for small-scale farmers.
- The absence of integrated machine learning techniques reduces prediction accuracy and limits the effectiveness of crop planning under changing climatic conditions.

These limitations highlight the need for an intelligent and automated crop recommendation system capable of analyzing multiple agricultural parameters and providing accurate, data-driven recommendations. The proposed **Smart Farmer** system addresses these challenges by integrating machine learning with modern web technologies to support farmers in selecting the most suitable crops, improving productivity, optimizing resource utilization, and promoting sustainable agricultural practices.

PROPOSED SYSTEM

The **Smart Farmer: AI-Based Smart Crop Recommendation System** is an intelligent web application developed to assist farmers in selecting the most suitable crop based on soil nutrients and climatic conditions. The system combines modern web technologies with machine learning to provide accurate, fast, and data-driven crop recommendations. Unlike traditional farming methods, which rely on experience and intuition, the proposed system analyzes agricultural parameters scientifically to support informed decision-making and improve crop productivity.

Overview of the Proposed System

The application enables users to register, log in securely, and access a crop recommendation module. Users provide seven agricultural parameters: **Nitrogen (N), Phosphorus (P), Potassium (K), soil pH, temperature, humidity, and rainfall**. These values are processed through a responsive user interface developed using **React.js** and **Tailwind CSS**.

The request is sent to the **Node.js** and **Express.js** backend, which validates the input and forwards it to the **Flask API**. The Flask service loads the trained **Extra Trees Classifier** model and predicts the most suitable crop. The prediction result includes the recommended crop, confidence score, crop category, water requirement, growth duration, market demand, and the top five recommended crops. User information and prediction history are stored in a **PostgreSQL** database for future reference.

Workflow of the Proposed System

The workflow begins with user authentication, followed by the submission of soil and climatic parameters through the recommendation form. The backend forwards the validated data to the Flask-based machine learning service for prediction. After processing, the prediction results are returned to the frontend and displayed to the user. Simultaneously, the prediction details are saved in the database, allowing users to review their recommendation history through the application dashboard.

Advantages of the Proposed System

- Provides accurate crop recommendations using the **Extra Trees Classifier**.
- Supports secure user registration, login, and prediction history management.
- Displays the **top five recommended crops** with confidence scores and crop details.
- Offers a responsive and user-friendly interface developed with **React.js** and **Tailwind CSS**.
- Uses a modular architecture with **React.js**, **Node.js**, **Express.js**, **Flask**, and **PostgreSQL**, making the system scalable and easy to maintain.
- Can be extended with future technologies such as weather APIs, IoT sensors, fertilizer recommendation, and crop disease detection.

SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

Category	Software / Tool	Purpose
Frontend	React.js	Building the responsive user interface
Styling	Tailwind CSS	Utility-first styling of UI components
Backend Runtime	Node.js	JavaScript runtime for server-side execution
Backend Framework	Express.js	REST API development and routing
ML Service	Flask	Serving the machine learning prediction API
Database	PostgreSQL	Relational data storage
IDE	Visual Studio Code	Source code editing and development
Programming Language	Python	Machine learning model development
Version Control	Git	Source code versioning and collaboration
API Testing	Postman	Testing and validating REST API endpoints

Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i5 (8th Generation) or equivalent
RAM	8 GB (16 GB recommended for model training)
Storage	256 GB SSD (with at least 10 GB free space)
Operating System	Windows 10 / 64-bit, Ubuntu 20.04 LTS, or macOS 12+
Internet Connectivity	Stable broadband connection for package installation and API testing

TECHNOLOGY DESCRIPTION

The **Smart Farmer: AI-Based Smart Crop Recommendation System** is developed using modern web technologies, a robust relational database, and a machine learning model. Each technology plays a significant role in ensuring the system is efficient, scalable, and user-friendly. The following sections describe the technologies used in the development of the application.

React.js

React.js is an open-source JavaScript library developed by **Meta** for building dynamic and interactive user interfaces. It follows a component-based architecture, allowing developers to build reusable UI components that improve code organization and maintainability.

In the Smart Farmer application, React.js is used to develop the entire frontend, including the landing page, user registration, login, dashboard, crop recommendation form, and result page. React efficiently updates the user interface using its Virtual DOM, providing a smooth user experience and faster rendering. It also simplifies communication with backend APIs and effectively manages application state, making the frontend responsive and easy to maintain.

Tailwind CSS

Tailwind CSS is a utility-first CSS framework that enables developers to create modern and responsive user interfaces using predefined utility classes. Unlike traditional CSS frameworks, Tailwind allows custom designs without writing extensive CSS code.

In this project, Tailwind CSS is used to design all application pages, including forms, navigation bars, dashboard cards, buttons, and recommendation results. It provides a clean, professional appearance while ensuring the application is fully responsive across desktops, tablets, and mobile devices. Tailwind CSS also speeds up frontend development by reducing the need for custom styling.

Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment built on Google's V8 JavaScript engine. It enables JavaScript to be executed outside the browser and is widely used for developing scalable server-side applications.

In the Smart Farmer system, Node.js serves as the backend runtime environment. It processes client requests, manages authentication, communicates with the PostgreSQL database, and interacts with the Flask machine learning service. Its asynchronous and event-driven architecture enables the application to efficiently handle multiple requests simultaneously.

Express.js

Express.js is a lightweight and flexible web application framework built on Node.js. It simplifies backend development by providing features such as routing, middleware, request handling, and REST API development.

In this project, Express.js manages user authentication, validates incoming requests, processes crop recommendation requests, and communicates with both the PostgreSQL database and the Flask prediction service. It acts as the central controller that coordinates all backend operations and ensures secure data transfer between different system components.

Flask

Flask is a lightweight Python web framework commonly used for deploying machine learning models as web services. Its simplicity and compatibility with Python's machine learning libraries make it an ideal choice for AI applications.

In the Smart Farmer application, Flask hosts the trained **Extra Trees Classifier** model. It receives input parameters from the Express.js backend, performs crop prediction, and returns the prediction results in JSON format. This separation of the machine learning service from the main backend improves modularity and simplifies future model updates.

PostgreSQL

PostgreSQL is a powerful, open-source relational database management system known for its reliability, security, and high performance. It efficiently stores structured data while maintaining data integrity through relational tables.

In this project, PostgreSQL stores user registration details, login information, and crop prediction records. It enables secure storage, efficient data retrieval, and proper management of application data. The database supports future scalability by allowing additional agricultural datasets and user information to be integrated seamlessly.

REST API

Representational State Transfer (REST) is a widely used architectural style for developing web services that communicate over the HTTP protocol. REST APIs provide standardized methods for exchanging data between different software components.

The Smart Farmer system uses REST APIs to establish communication between the React.js frontend, Node.js and Express.js backend, Flask prediction service, and PostgreSQL database. Data is exchanged in JSON format, enabling efficient and secure interaction among all application modules.

Extra Trees Classifier

The **Extra Trees (Extremely Randomized Trees) Classifier** is an ensemble machine learning algorithm that builds multiple decision trees using randomly selected features and split points. The final prediction is determined through majority voting among all trees, resulting in improved accuracy and reduced overfitting.

The Smart Farmer application uses the Extra Trees Classifier as its core prediction model. The model is trained using agricultural data containing soil nutrients and climatic parameters. Based on user inputs such as Nitrogen, Phosphorus, Potassium, soil pH, temperature, humidity, and rainfall, it predicts the most suitable crop along with a confidence score. The model was selected because of its high prediction accuracy, computational efficiency, and ability to handle complex agricultural datasets.

Joblib

Joblib is a Python library used for serializing and deserializing machine learning models. It enables trained models to be saved to disk and loaded later without retraining.

In the Smart Farmer project, Joblib is used to save the trained Extra Trees Classifier as a `.pkl` file. During application execution, the Flask API loads the model from this file and performs crop predictions in real time. This approach significantly reduces prediction time and improves application performance.

Axios

Axios is a promise-based HTTP client for JavaScript that simplifies sending asynchronous HTTP requests and handling server responses.

In the Smart Farmer frontend, Axios is used to communicate with the Express.js backend APIs. It sends user registration, login, and crop prediction requests, receives JSON responses, and handles loading states and error messages efficiently. Axios ensures reliable communication between the frontend and backend, contributing to a smooth and responsive user experience.

DATABASE DESIGN

Database Schema Overview

The Smart Farmer application uses a PostgreSQL relational database to persist two primary categories of data: user account information and crop prediction history. The schema is designed with normalization principles in mind, ensuring data integrity and minimizing redundancy while maintaining efficient query performance for common access patterns such as retrieving a user's prediction history.

Users Table

Column Name	Data Type	Description
user_id	SERIAL (Primary Key)	Unique identifier for each user
full_name	VARCHAR(100)	Full name of the registered user
email	VARCHAR(150), UNIQUE	User's email address, used for login
password_hash	VARCHAR(255)	Securely hashed user password
created_at	TIMESTAMP	Timestamp of account creation

Recommendation History Table

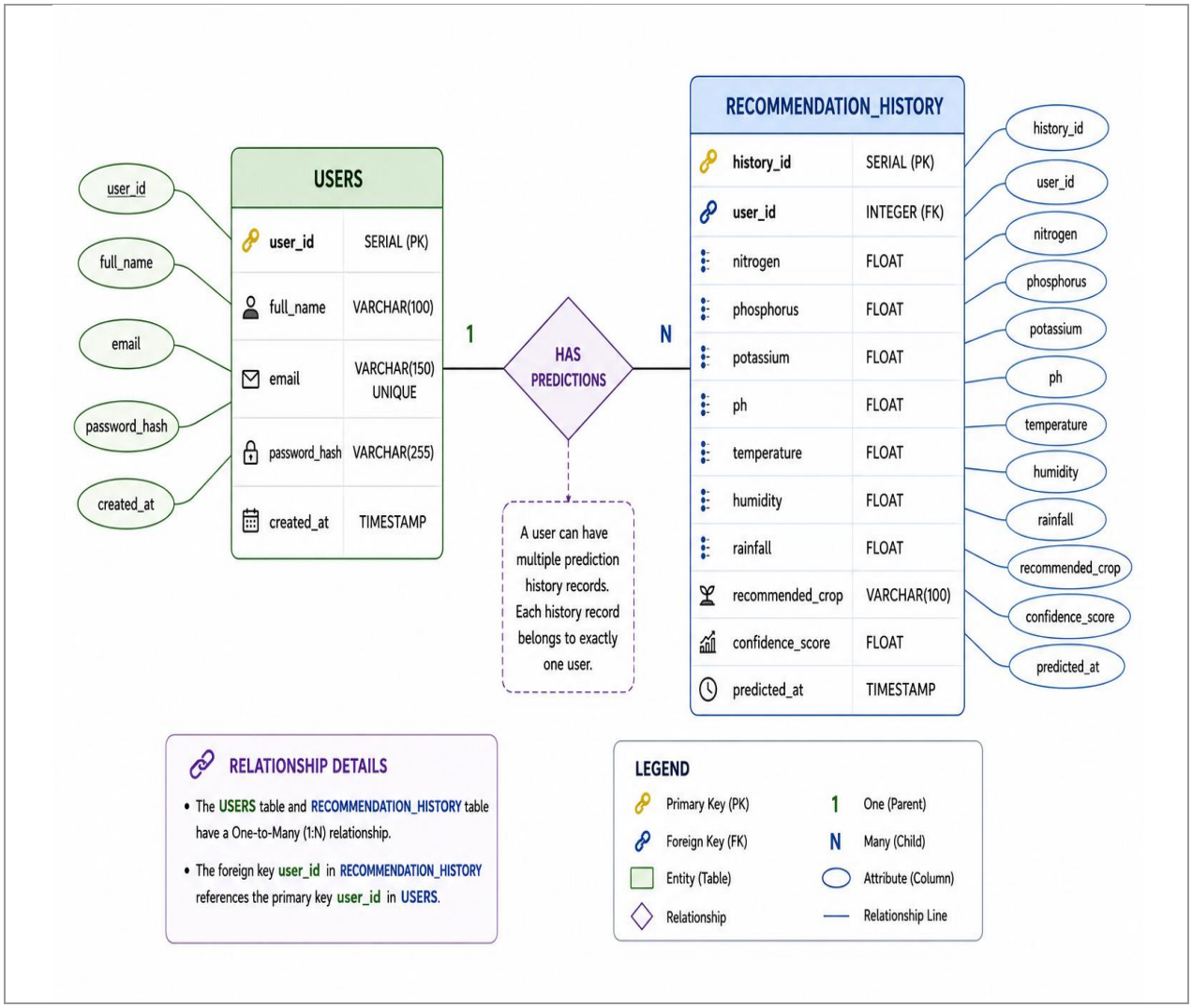
Column Name	Data Type	Description
history_id	SERIAL (Primary Key)	Unique identifier for each prediction record
user_id	INTEGER (Foreign Key)	References user_id in the Users table
nitrogen	FLOAT	Nitrogen value entered by the user
phosphorus	FLOAT	Phosphorus value entered by the user
potassium	FLOAT	Potassium value entered by the user
ph	FLOAT	Soil pH value entered by the user
temperature	FLOAT	Temperature value entered by the user
humidity	FLOAT	Humidity value entered by the user
rainfall	FLOAT	Rainfall value entered by the user
recommended_crop	VARCHAR(100)	Crop predicted by the machine learning model
confidence_score	FLOAT	Model's confidence score for the prediction

Column Name	Data Type	Description
predicted_at	TIMESTAMP	Timestamp when the prediction was made

Table Relationships

The Users table and the Recommendation History table share a one-to-many relationship, whereby a single user may have multiple associated prediction history records, while each history record belongs to exactly one user. This relationship is enforced through a foreign key constraint on the user_id column of the Recommendation History table, which references the primary key of the Users table, ensuring referential integrity across the database.

Entity-Relationship Diagram



MACHINE LEARNING MODEL

Dataset

The machine learning component of the **Smart Farmer** system is trained on an agricultural dataset comprising records of **soil nutrient levels (Nitrogen, Phosphorus, Potassium)**, **soil pH**, and **climatic parameters (Temperature, Humidity, Rainfall)**, each labeled with the corresponding **crop** that is most suitable for cultivation under those conditions. The dataset spans a wide range of **crop categories**, ensuring the trained model generalizes well across diverse agronomic scenarios rather than overfitting to a narrow subset of crops.

Data Preprocessing

Prior to model training, the dataset undergoes a preprocessing pipeline that includes **handling missing values**, **removing duplicate records**, and **verifying data type consistency** across all numerical features. **Categorical crop labels** are encoded appropriately for use with the **Scikit-learn classification pipeline**, and numerical features are examined for **outliers** that could adversely affect model performance.

Feature Selection

The seven agronomic parameters, namely **Nitrogen**, **Phosphorus**, **Potassium**, **Temperature**, **Humidity**, **Soil pH**, and **Rainfall**, were selected as the model's input features based on their established agronomic relevance to **crop suitability**. These features capture both the **soil nutrient composition** and the **climatic conditions**, enabling the model to learn meaningful relationships between environmental factors and crop categories.

Train-Test Split

The preprocessed dataset is divided into **Training (80%)** and **Testing (20%)** subsets to evaluate the model's performance on unseen data. **Stratified sampling** is applied to preserve the distribution of each crop class across both datasets, ensuring unbiased model evaluation.

Extra Trees Algorithm

The **Extra Trees Classifier (Extremely Randomized Trees)** is an **ensemble machine learning algorithm** that constructs multiple **decision trees** using randomly selected **features** and **split thresholds**. Unlike the **Random Forest** algorithm, Extra Trees introduces greater randomness during tree construction, which helps reduce **variance**, minimize **overfitting**, and improve computational efficiency. The final prediction is obtained through **majority voting** across all decision trees in the ensemble.

Mathematical Formulation

For a given node, the **Extra Trees Classifier** randomly selects candidate features and evaluates the quality of each split using the **Gini Index**.

Gini Index

$$Gini(D) = 1 - \sum_{i=1 \text{ to } c} (p_i)^2$$

where:
D = Dataset at the current node
c = Number of crop classes
$\mathbf{p_i}$ = Probability of class *i*

The final prediction of the ensemble is obtained using **Majority Voting**.

$$\hat{y} = \text{mode } \{T1(x), T2(x), ..., Tn(x)\}$$

where:
$\mathbf{T_1 \ldots T_n}$ = Individual decision trees
x = Input feature vector

Hyperparameter Tuning

The performance of the **Extra Trees Classifier** depends on several **hyperparameters**, including:

- **Number of Estimators**
- **Maximum Tree Depth**
- **Minimum Samples Split**
- **Minimum Samples Leaf**

These parameters were optimized using **Grid Search** combined with **K-Fold Cross Validation** to obtain the best model performance while reducing **overfitting**.

Model Evaluation

The trained model was evaluated using standard **classification metrics**, including:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **Confusion Matrix**

These metrics provide a comprehensive assessment of the model's predictive performance across different crop classes.

Accuracy

The final **Extra Trees Classifier** achieved an overall **accuracy of approximately 95%** on the testing dataset. This high accuracy demonstrates the model's ability to provide reliable crop recommendations based on **soil nutrients** and **climatic conditions**, making it suitable for deployment in the **Smart Farmer** web application.

Model Serialization Using Joblib

After training and validation, the **Extra Trees Classifier** was serialized using the **Joblib** library and saved as a `.pkl` file. Model serialization eliminates the need for retraining during runtime and significantly improves **prediction speed** and **application performance**.

Flask Model Integration

The **Flask API** loads the serialized **Extra Trees** model during application startup using **Joblib**. It exposes a **POST REST API endpoint** that accepts the seven agronomic parameters in **JSON** format. The Flask service generates the prediction and returns a structured response containing the **recommended crop**, **confidence score**, **Top 5 crop recommendations**, **crop category**, **water requirement**, **growth duration**, and **market demand**, which are displayed in the **React.js** frontend.

SOFTWARE IMPLEMENTATION

Frontend Implementation Using React.js and Tailwind CSS

The frontend of the **Smart Farmer** application is developed using **React.js** and **Tailwind CSS** to provide a responsive, interactive, and user-friendly interface. React.js follows a **component-based architecture**, enabling the application to be divided into reusable modules such as the **Landing Page**, **User Registration**, **Login**, **Dashboard**, **Crop Recommendation Form**, and **Recommendation Result** page. **React Hooks** (`useState`, `useEffect`) are used to manage application state and API responses, while **React Router** handles page navigation. **Axios** is used for communication with the backend. Tailwind CSS provides responsive layouts, consistent styling, and cross-device compatibility.

Key Features:

- User Registration and Login
- Landing Page
- Dashboard
- Crop Recommendation Form

- Recommendation Result Page
- Responsive UI
- API Integration using Axios

Backend Implementation Using Node.js and Express.js

The backend is developed using **Node.js** and **Express.js**, which provide RESTful APIs for user authentication, request processing, and communication with the database and machine learning service. The backend validates user inputs, processes crop recommendation requests, stores prediction records, and returns structured JSON responses. Middleware is implemented for **request validation**, **CORS**, and **error handling**, ensuring secure and reliable communication.

Database Implementation Using PostgreSQL

The application uses **PostgreSQL** as its relational database management system to store user information and prediction history. The database consists of two primary tables: **Users** and **Prediction History**. Primary and foreign key constraints maintain data integrity, while connection pooling and indexing improve database performance and query efficiency.

Flask API Implementation

The machine learning service is implemented using **Flask**, which loads the trained **Extra Trees Classifier** model using **Joblib** during application startup. The Flask API accepts the seven agricultural input parameters in **JSON** format, performs crop prediction, and returns the **recommended crop**, **confidence score**, **crop category**, **water requirement**, **growth duration**, **market demand**, and **top five recommended crops**.

Communication Between System Components

The Smart Farmer application follows a **client-server architecture**. The **React.js** frontend collects user input and sends it to the **Node.js and Express.js** backend using **Axios**. After validating the request, the backend forwards the data to the **Flask API**, where the **Extra Trees Classifier** generates the crop recommendation. The prediction results are stored in the **PostgreSQL** database and returned to the frontend for display. This architecture ensures **efficient communication**, **scalability**, **security**, and **maintainability** of the application.

SOFTWARE CODING DEMONSTRATION

This chapter presents representative code snippets from each major layer of the Smart Farmer application, accompanied by explanations of their functionality and role within the overall system.

14.1 React Component

The following snippet shows a simplified version of the Crop Recommendation Form component, which captures the seven agronomic input parameters from the user.

```
function CropRecommendationForm() {
  const [formData, setFormData] = useState({
    nitrogen: '', phosphorus: '', potassium: '',
    temperature: '', humidity: '', ph: '', rainfall: ''
  });

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const result = await getCropRecommendation(formData);
    setPrediction(result);
  };

  return ( <form onSubmit={handleSubmit}> ... </form> );
}
```

The component maintains form state through the useState hook, updates individual fields via a shared handleChange handler, and triggers the prediction workflow through handleSubmit, which delegates the API call to a dedicated service function.

Tailwind CSS Styling

The following markup illustrates the use of Tailwind utility classes to style an input field within the recommendation form.

```
<input
  type="number"
  name="nitrogen"
  className="w-full px-4 py-2 border border-gray-300 rounded-lg
             focus:ring-2 focus:ring-green-500 focus:outline-none"
  placeholder="Enter Nitrogen (N)"
  onChange={handleChange}
/>
```

Utility classes such as px-4, py-2, and rounded-lg control padding and border radius, while the focus: prefix applies conditional styling when the input is focused, providing clear visual feedback to the user.

Axios API Call

The service function below encapsulates the Axios call used to request a crop recommendation from the backend.

```
export const getCropRecommendation = async (formData) => {
  const response = await axios.post(
    '/api/recommendation/predict',
    formData,
    { headers: { Authorization: `Bearer ${getToken()}` } }
  );
  return response.data;
};
```

This function sends the collected form data as a JSON payload to the backend's prediction endpoint, attaching the user's authentication token in the request header to identify the requesting user.

Express Route

```
const router = require('express').Router();
const { predictCrop } = require('../controllers/recommendationController');
const authMiddleware = require('../middleware/authMiddleware');

router.post('/predict', authMiddleware, predictCrop);

module.exports = router;
```

This route definition maps incoming POST requests at /predict to the predictCrop controller function, after first passing the request through the authMiddleware to verify the user's authentication status.

Express Controller

```
exports.predictCrop = async (req, res) => {
  try {
    const { nitrogen, phosphorus, potassium,
            temperature, humidity, ph, rainfall } = req.body;

    const mlResponse = await axios.post(
      process.env.FLASK_API_URL + '/predict',
      { nitrogen, phosphorus, potassium, temperature, humidity, ph, rainfall }
    );

    await pool.query(
      `INSERT INTO recommendation_history
       (user_id, nitrogen, phosphorus, potassium, ph,
        temperature, humidity, rainfall, recommended_crop, confidence_score)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)`,
      [req.user.id, nitrogen, phosphorus, potassium, ph,
       temperature, humidity, rainfall,
       mlResponse.data.crop, mlResponse.data.confidence]
    );

    res.status(200).json(mlResponse.data);
  } catch (error) {
    res.status(500).json({ message: 'Prediction failed', error: error.message });
  }
};
```

The controller extracts the agronomic parameters from the request body, forwards them to the Flask prediction service, persists the resulting recommendation to the PostgreSQL history table, and finally returns the prediction results to the client.

Authentication Middleware

```
module.exports = function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token provided' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
};
```

This middleware verifies the JSON Web Token (JWT) supplied in the Authorization header of each protected request, attaching the decoded user information to the request object for downstream use, or rejecting the request if the token is missing or invalid.

Flask Prediction API

```
@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    features = extract_features(data)
    result = predict_crop(features)
    return jsonify(result), 200
```

This Flask route accepts a JSON payload, extracts and validates the input features, delegates the prediction logic to a helper function, and returns the resulting recommendation as a JSON response.

Machine Learning Prediction Function

```
def predict_crop(features):
    input_array = np.array(features).reshape(1, -1)
    scaled_input = scaler.transform(input_array)

    probabilities = model.predict_proba(scaled_input)[0]
    top5_idx = np.argsort(probabilities)[-5:][::-1]

    return {
        'crop': label_encoder.inverse_transform([top5_idx[0]])[0],
        'confidence': float(probabilities[top5_idx[0]]),
        'top5': [
            {
                'crop': label_encoder.inverse_transform([i])[0],
                'confidence': float(probabilities[i])
            } for i in top5_idx
        ]
    }
```

This function transforms the raw input features using the previously fitted scaler, obtains class probabilities from the trained Extra Trees model, and identifies the top five crop classes with the highest predicted probabilities, returning them along with their respective confidence scores.

PostgreSQL Connection

```
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

module.exports = pool;
```

This module configures and exports a PostgreSQL connection pool using environment variables for database credentials, allowing the connection pool to be reused across all controller modules that require database access.

Database Query Example

```
exports.getHistory = async (req, res) => {
  const result = await pool.query(
    `SELECT * FROM recommendation_history
     WHERE user_id = $1
     ORDER BY predicted_at DESC`,
    [req.user.id]
  );
  res.status(200).json(result.rows);
};
```

This query retrieves all prediction history records belonging to the authenticated user, ordered by the most recent prediction first, and returns them as a JSON array to populate the history module of the frontend.

OUTPUT SCREENSHOTS

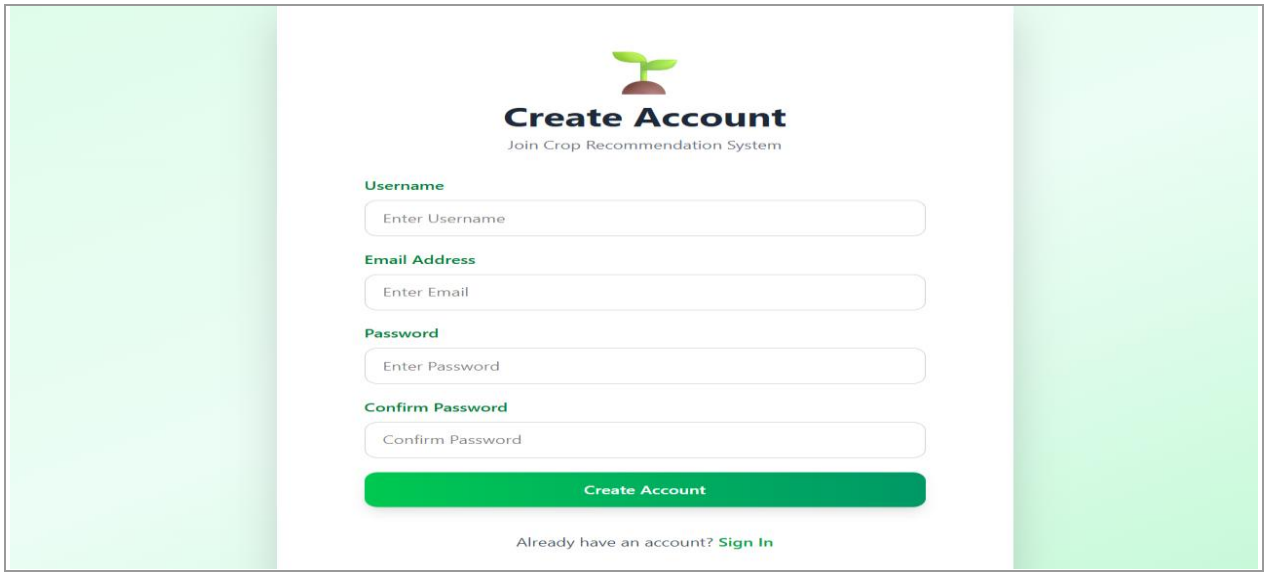
This chapter presents placeholders for the key screens of the Smart Farmer application, each accompanied by a description of its purpose and functionality. The actual screenshots should be captured from the running application and inserted in place of the placeholders below.

Landing Page

The landing page serves as the entry point of the application, introducing the Smart Farmer platform to new visitors and providing navigation options to register or log in. It is designed with a clean, agriculture-themed visual identity to establish immediate context for the user.

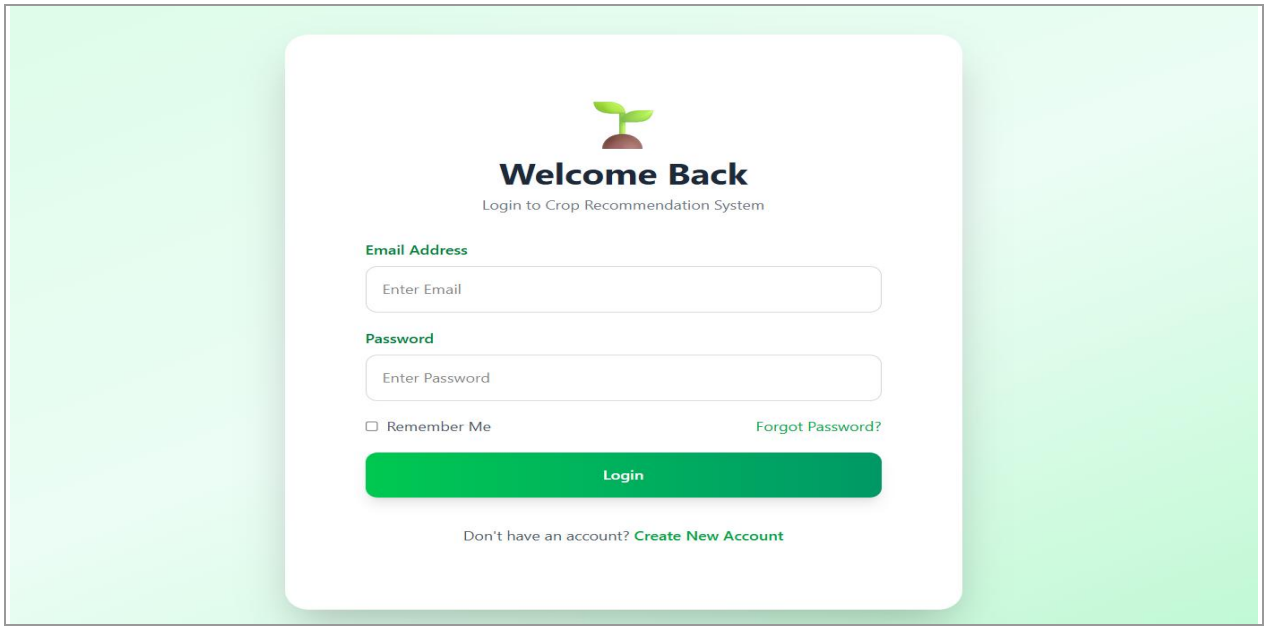
Registration Page

The registration page allows new users to create an account by providing their full name, email address, and password. Client-side validation ensures that all required fields are correctly filled before the request is submitted to the backend.



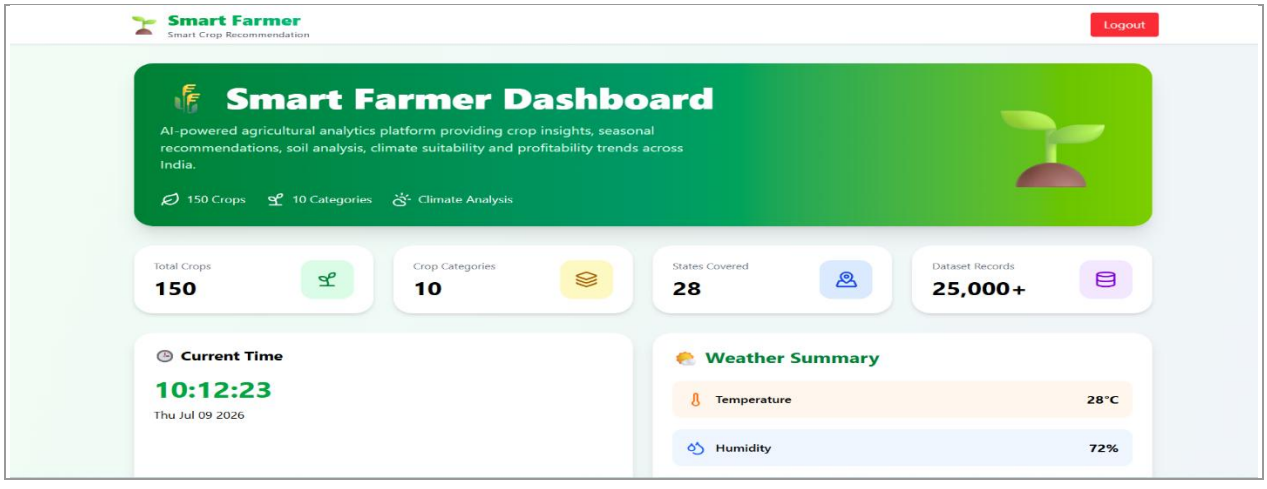
Login Page

The login page authenticates returning users by verifying their email and password against the records stored in the PostgreSQL database, issuing a JSON Web Token upon successful authentication to maintain the user's session.



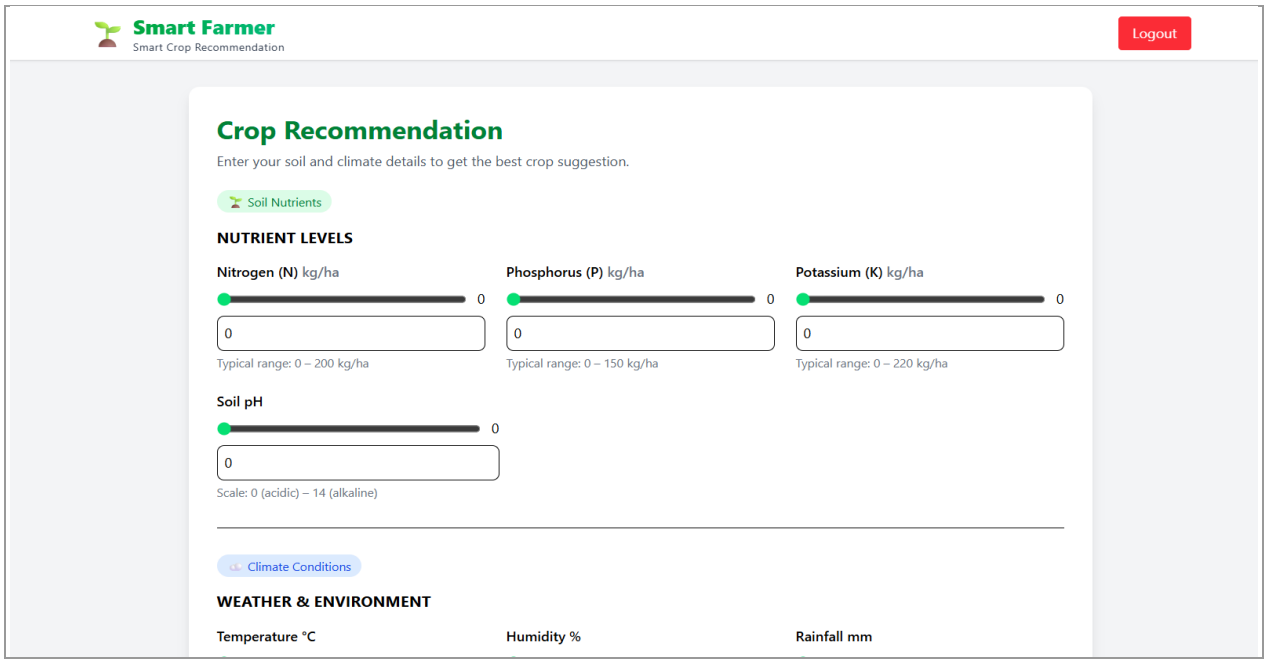
Dashboard

The dashboard presents a summary view of the user's account, including quick-access links to the recommendation form and a snapshot of recent prediction activity, serving as the central hub of the application post-login.



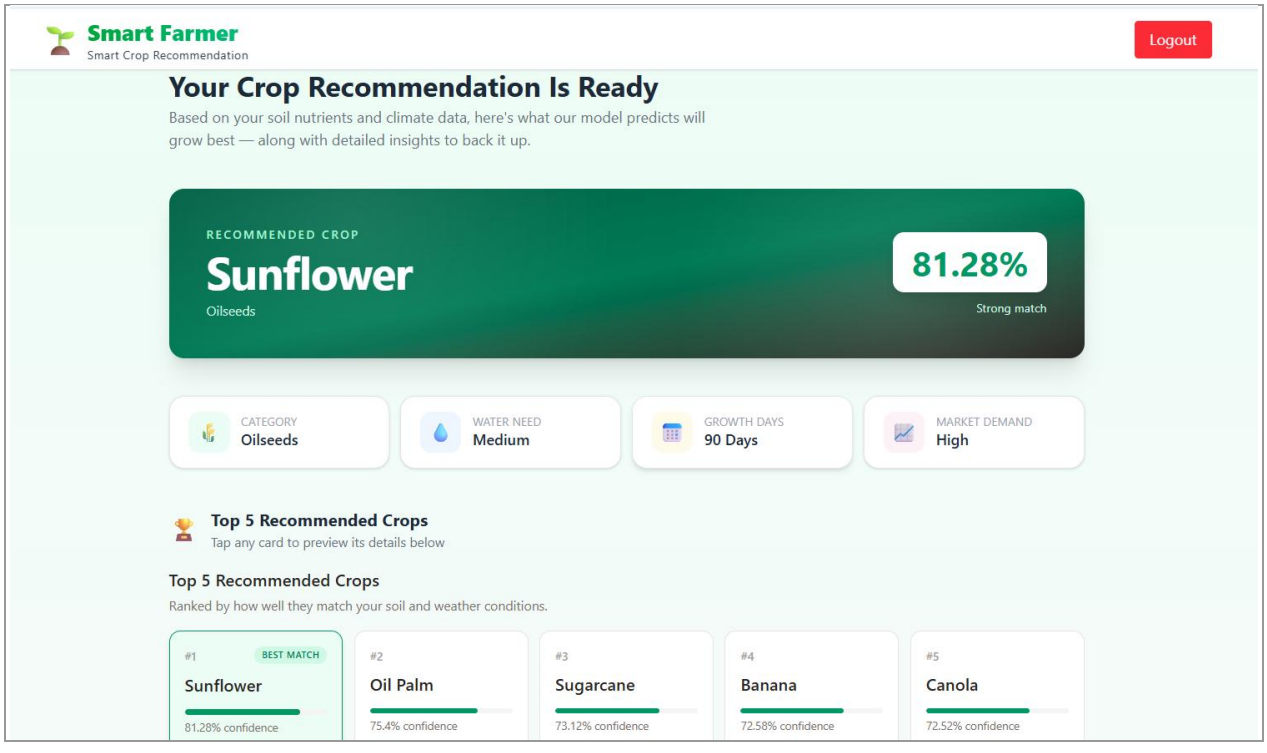
Crop Recommendation Form

This page presents the structured input form through which users enter the seven agronomic parameters required for crop prediction, with inline validation to guide correct data entry.



Prediction Result

The prediction result page displays the primary recommended crop along with its confidence score, water requirement, growth duration, market demand, and crop category, presenting the machine learning model's output in a clear and actionable format.



ADVANTAGES

The **Smart Farmer: AI-Based Smart Crop Recommendation System** offers several advantages over traditional crop selection methods by combining machine learning with modern web technologies. The major advantages of the proposed system are as follows:

1. Provides **accurate, data-driven crop recommendations** based on soil nutrients and climatic conditions.
2. Utilizes the **Extra Trees Classifier**, achieving high prediction accuracy and reliable recommendations.
3. Generates **instant crop predictions**, enabling farmers to make timely agricultural decisions.
4. Recommends the **top five suitable crops** along with confidence scores for better decision-making.
5. Displays additional crop information such as **water requirement, growth duration, crop category, and market demand**.
6. Implements **secure user authentication** to protect user accounts and application data.
7. Offers a **responsive and user-friendly interface** developed using **React.js** and **Tailwind CSS**, supporting both desktop and mobile devices.
8. Uses a **modular architecture** with **React.js, Node.js, Express.js, Flask, and PostgreSQL**, making the system scalable and easy to maintain.
9. Reduces dependence on traditional farming practices by providing **scientific and AI-based recommendations**.
10. Supports future enhancements such as **real-time weather integration, IoT-based soil monitoring, fertilizer recommendation, and crop disease detection**.

LIMITATIONS

Although the **Smart Farmer: AI-Based Smart Crop Recommendation System** provides accurate and intelligent crop recommendations, it has certain limitations that can be addressed in future enhancements.

- The machine learning model is trained on a **fixed agricultural dataset** and does not incorporate **real-time weather** or **satellite data**, which may affect prediction accuracy under changing environmental conditions.
- Users must **manually enter** soil and climatic parameters, which may introduce input errors and affect the quality of recommendations.
- The system can recommend only the **crop varieties available in the training dataset**, limiting its applicability to region-specific or newly introduced crops.
- The recommendation process does not consider **economic factors** such as cultivation cost, market fluctuations, labor availability, or government subsidies.
- The application currently supports **only a single language**, which may reduce accessibility for farmers who prefer regional languages.

- The system does not analyze **seasonal variations** or historical weather patterns beyond the user-provided input values.
- The application has been evaluated in a **development environment** and has not yet been tested under large-scale real-world deployment with a high number of concurrent users.

Despite these limitations, the Smart Farmer system provides a strong foundation for developing an intelligent and scalable crop recommendation platform. Future enhancements can further improve its accuracy, usability, and practical applicability in precision agriculture.

FUTURE ENHANCEMENTS

The **Smart Farmer: AI-Based Smart Crop Recommendation System** provides a strong foundation for intelligent agricultural decision-making. However, the system can be further enhanced by integrating advanced technologies and additional features to improve its accuracy, usability, and practical applicability. The following enhancements are proposed for future development.

Real-Time Weather Integration

The system can be integrated with **real-time weather APIs** to automatically retrieve temperature, humidity, and rainfall data based on the user's location. This will reduce manual data entry and improve the accuracy of crop recommendations.

Satellite and Remote Sensing Integration

Satellite imagery and remote sensing technologies can be incorporated to analyze **soil moisture**, **vegetation health**, and land conditions. These additional inputs will enhance the performance of the crop recommendation model.

Fertilizer Recommendation

A fertilizer recommendation module can be developed to suggest the appropriate **fertilizer type**, **quantity**, and **application schedule** based on soil nutrient deficiencies, helping farmers improve crop productivity while minimizing fertilizer wastage.

Crop Disease Detection

Computer vision techniques can be integrated to detect crop diseases using images of leaves uploaded by farmers. This feature will enable **early disease identification** and provide suitable treatment recommendations.

Mobile Application

A dedicated **Android** and **iOS** mobile application can be developed to make the system more accessible for farmers, allowing them to receive crop recommendations directly through their smartphones.

Voice-Based Assistance

Voice-enabled interaction can be implemented to support farmers who are less familiar with digital applications. Voice commands and audio responses will improve usability and accessibility.

Multi-Language Support

The application can support multiple regional languages such as **Tamil**, **Hindi**, **Telugu**, and **Kannada**, making the system easier to use for farmers across different regions of India.

Government Scheme Integration

The platform can be integrated with government agricultural portals to provide information about **subsidies**, **loan schemes**, **crop insurance**, and other farmer welfare programs relevant to the recommended crop.

Market Price Prediction

A market price prediction module can be incorporated to estimate future crop prices using historical market data. This will help farmers choose crops based on both agricultural suitability and expected profitability.

IoT-Based Smart Farming

The system can be integrated with **IoT-based soil sensors** to automatically collect real-time data on **soil nutrients**, **soil moisture**, **temperature**, and **humidity**. Automated data collection will improve recommendation accuracy while eliminating manual input.

Overall, these enhancements will transform the Smart Farmer application into a comprehensive **smart agriculture platform**, supporting precision farming, sustainable resource management, and data-driven agricultural decision-making. By integrating emerging technologies such as **Artificial Intelligence**, **Internet of Things (IoT)**, **Computer Vision**, and **Cloud Services**, the system can provide greater value to farmers and contribute to the advancement of modern agriculture.

CONCLUSION

The **Smart Farmer: AI-Based Smart Crop Recommendation System** successfully demonstrates the integration of **machine learning** with modern **full-stack web technologies** to provide an intelligent and efficient solution for crop recommendation. The system analyzes essential soil nutrients and climatic parameters, including **Nitrogen, Phosphorus, Potassium, soil pH, temperature, humidity, and rainfall**, to recommend the most suitable crop using the **Extra Trees Classifier** algorithm.

The application is developed using **React.js** and **Tailwind CSS** for the frontend, **Node.js** and **Express.js** for the backend, **Flask** for machine learning model deployment, and **PostgreSQL** for secure data storage. The modular architecture ensures efficient communication between different system components while providing scalability, maintainability, and improved system performance.

The trained **Extra Trees Classifier** achieved high prediction accuracy, enabling the system to generate reliable crop recommendations along with confidence scores, crop category, water requirements, growth duration, market demand, and the top five recommended crops. These features help farmers make informed agricultural decisions, optimize resource utilization, and improve crop productivity.

The project also demonstrates the practical application of **Artificial Intelligence** in agriculture by transforming complex agricultural data into meaningful recommendations through a user-friendly web application. Although the current implementation has certain limitations, its flexible architecture provides a strong foundation for future enhancements, including **real-time weather integration**, **IoT-based soil monitoring**, **fertilizer recommendation**, **crop disease detection**, **market price prediction**, and **multilingual support**.

In conclusion, the **Smart Farmer** system represents an effective combination of **Artificial Intelligence**, **Machine Learning**, and **Full-Stack Web Development** to support **precision agriculture**. By providing accurate, data-driven crop recommendations, the application contributes toward **sustainable farming**, improved agricultural productivity, and better decision-making, demonstrating the potential of intelligent technologies in modern agriculture.